INTERMEDIATE PHASE · SESSION 13 OF 30

Day 13
Iterators & Generators

2-Hour Live Session · Instructor Guide

■ Duration	■ Prerequisites	■ Difficulty
2 hours	Days 10–12 (OOP, Modules)	Core / Intermediate

1 · Session Overview

Goal: Students understand how Python's iteration protocol works under the hood, can build their own iterator classes, write generator functions with `yield`, and use generator expressions and the `itertools` module to write memory-efficient, lazy-evaluated code.

Outcome: By the end of class, students will have built a custom iterator for a Mumbai delivery route, written generator functions for processing large order logs lazily, and used `itertools.chain`, `islice`, `product`, and `cycle` in real scenarios.

Timeline

Time	Segment
0:00 – 0:15	Recap Day 12 (modules/packages) + why iteration matters
0:15 – 0:40	The iterator protocol — __iter__ / __next__, build RouteIterator
0:40 – 1:05	Generator functions — yield, generator state, StopIteration
1:05 – 1:15	■ Break
1:15 – 1:35	Generator expressions vs list comprehensions
1:35 – 1:55	itertools — chain, islice, product, cycle
1:55 – 2:00	Wrap-up, mini-challenge reveal, Day 14 preview

■■■ Instructor Note

This is a conceptually dense day. Lean hard on the 'lazy evaluation = don't compute until asked' mental model. The delivery-route and order-log examples should make the abstraction concrete — keep returning to them whenever a student looks lost.

2 · Prerequisites & Preparation

Requirements

Requirement	Detail
Python version	3.12 (installed Day 1)
Prior concepts	Classes & __init__ (Day 10), inheritance & dunder methods (Day 11), modules (Day 12)
VS Code	Open with Day 13 starter folder
No new installs	itertools is part of the standard library

Instructor Pre-Session Checklist

✔	Test all four .py concept files run cleanly end-to-end
✔	Have the 'delivery route' diagram ready (whiteboard or slide)
✔	Pre-load the order_log.txt-style data used in generator examples (in-code, no file needed)
✔	Review the difference between StopIteration being raised manually vs automatically

■ Common Setup Pitfall

Students sometimes confuse 'iterable' and 'iterator'. Emphasize: every iterator is an iterable, but not every iterable is an iterator (a list is iterable but is not itself an iterator).

3 · Step-by-Step Instruction Guide

STEP 1 — The Iterator Protocol (__iter__ / __next__)

What to Say

"Every time you write a for loop, Python is quietly calling iter() on your object to get an iterator, then calling next() on it over and over until it raises StopIteration. Today we build that machinery ourselves — using a delivery route across Mumbai as our example."

route_iterator.py

```
class DeliveryRoute:
    """Iterable: a sequence of delivery stops in Mumbai."""
    def __init__(self, stops):
        self.stops = stops

    def __iter__(self):
        return RouteIterator(self.stops)

class RouteIterator:
    """The actual iterator: tracks position + produces next stop."""
    def __init__(self, stops):
        self.stops = stops
        self.index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.index >= len(self.stops):
            raise StopIteration
        stop = self.stops[self.index]
        self.index += 1
        return stop

route = DeliveryRoute(["Andheri", "Bandra", "Worli", "Colaba"])
for stop in route:
    print(stop)
# Output:
# Andheri
# Bandra
# Worli
# Colaba
```

Error	Cause / Fix
TypeError: 'X' object is not iterable	Class is missing __iter__
TypeError: iter() returned non-iterator	__iter__ didn't return an object with __next__

Infinite loop	Forgot to raise StopIteration inside __next__

■ Done When

Students can explain, in their own words, what iter() and next() each do, and have run the RouteIterator example successfully.

STEP 2 — Generator Functions (yield)

What to Say

"Writing __iter__ and __next__ by hand works, but it's verbose. Python gives us a shortcut: generator functions. Any function with yield in it automatically becomes a generator — Python builds the iterator machinery for you."

generators_basics.py

```
def delivery_route_gen(stops):
    """Generator version — same behaviour, far less code."""
    for stop in stops:
        yield stop

route = delivery_route_gen(["Andheri", "Bandra", "Worli"])
print(type(route))            # Output: <class 'generator'>
print(next(route))            # Output: Andheri
print(next(route))            # Output: Bandra

def order_amounts(orders):
    """Lazily yields only orders above 500 rupees."""
    for order in orders:
        if order > 500:
            yield order

orders = [250, 720, 150, 980, 60]
for amt in order_amounts(orders):
    print(f"■{amt}")
# Output:
# ■720
# ■980
```

■ Key Idea — Lazy Evaluation

A generator does NOT compute all values upfront. Each call to next() runs the function until the next yield, then pauses — state and all. This is what makes generators memory-efficient for huge datasets.

STEP 3 — Generator Expressions

What to Say

"A generator expression looks exactly like a list comprehension, but with round brackets instead of square ones. The difference: it doesn't build the whole list in memory — it produces one value at a time, on demand."

generator_expressions.py

```
prices = [499, 1299, 250, 899, 1999]

# List comprehension — builds the FULL list in memory immediately
discounted_list = [p * 0.9 for p in prices]
print(discounted_list)
# Output: [449.1, 1169.1, 225.0, 809.1, 1799.1]

# Generator expression — values produced lazily, one at a time
discounted_gen = (p * 0.9 for p in prices)
print(discounted_gen)
# Output: <generator object <genexpr> at 0x...>
print(next(discounted_gen))   # Output: 449.1
print(sum(discounted_gen))    # consumes the REST: 4002.2 (4 remaining)
```

Error	Cause / Fix
Generator already exhausted	Generators can only be consumed ONCE — re-create if needed again
Confusing () with generator call	(x for x in y) is a genexpr; tuple(x for x in y) makes an actual tuple

STEP 4 — The itertools Module

What to Say

"itertools is the Python standard library's toolbox for building iterators efficiently. We'll use four of the most common: chain, islice, product, and cycle."

itertools_demo.py

```
from itertools import chain, islice, product, cycle

# chain — join several iterables into one continuous stream
mumbai_stops = ["Andheri", "Bandra"]
delhi_stops = ["Saket", "Dwarka"]
all_stops = list(chain(mumbai_stops, delhi_stops))
print(all_stops)
# Output: ['Andheri', 'Bandra', 'Saket', 'Dwarka']

# islice — take a lazy slice of an iterator without exhausting it all
def all_orders():
    n = 1
    while True:
        yield n * 111
        n += 1

first_five = list(islice(all_orders(), 5))
print(first_five)   # Output: [111, 222, 333, 444, 555]

# product — Cartesian product (like nested for loops)
sizes = ["S", "M", "L"]
colors = ["Red", "Blue"]
for combo in product(sizes, colors):
    print(combo)
# Output: ('S','Red') ('S','Blue') ('M','Red') ...

# cycle — repeat an iterable forever (use with islice to stay finite!)
delivery_riders = cycle(["Riya", "Sanjay", "Arjun"])
assigned = list(islice(delivery_riders, 7))
print(assigned)
# Output: ['Riya','Sanjay','Arjun','Riya','Sanjay','Arjun','Riya']
```

■ Watch Out — Infinite Iterators

cycle() and count() never stop on their own. ALWAYS pair them with islice() or a break condition, or the program will hang/run forever.

■ Done When

Students have successfully run all four itertools examples and can describe what each function is used for in one sentence.

4 · Session Wrap-Up

Student Checklist

✔	I can explain the iterator protocol (__iter__ / __next__)
✔	I can write a generator function using yield
✔	I understand why generators are memory-efficient (lazy evaluation)
✔	I can write a generator expression and explain how it differs from a list comprehension
✔	I can use chain, islice, product, and cycle from itertools

Key Concepts Recap

Iterators are objects with __iter__ and __next__ that produce one value at a time and raise StopIteration when exhausted. Generator functions (using yield) and generator expressions are concise shortcuts that build this machinery automatically. The itertools module provides battle-tested, memory-efficient tools for combining and slicing iterators — chain, islice, product, and cycle being the most commonly used.

■ Next Day Preview — Day 14: Decorators & Closures

Tomorrow we look at first-class functions, closures, and how to write a @decorator from scratch — plus the built-in @staticmethod, @classmethod, and @property decorators.

■ Troubleshooting Notes

If a generator seems to 'return nothing', remind students that calling the generator function doesn't run any code — it only runs when next() is called or it's iterated over.

5 · Quick Reference Card

Daily Workflow Commands

```terminal
uv venv && source .venv/bin/activate     # activate environment
python day13_01_iterators_basics.py       # run a concept file
python day13_practise_problems_01_iterators.py
```

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + `	Open integrated terminal
F5	Run/debug current file
Ctrl/Cmd + /	Toggle line comment
Shift + Alt + F	Format document

Tool / Concept Summary

Concept	Use it when...
__iter__ / __next__	You need full custom control over a custom iterable class
yield (generator function)	You want lazy iteration with minimal boilerplate
Generator expression (x for x in y)	You'd write a list comprehension but don't need the full list in memory
itertools.chain	Combining multiple iterables into one stream
itertools.islice	Taking a slice of a (possibly infinite) iterator
itertools.product	Generating all combinations (replaces nested for loops)
itertools.cycle	Repeating a small sequence indefinitely (pair with islice!)